CSA 1737 – ARTIFICIAL INTELLIGENCE (Prolog)

Laboratory Record – Experiments 36 to 40

HOW TO RUN THE PROLOG PROGRAMS IN GOOGLE COLAB

Google Colab does not have SWI-Prolog by default. Install it once, then write each program into a file with %%writefile and run it with swipl. Each program ends with ":- initialization(main)." so the queries run automatically.

Cell 1 (run once):

```
!apt-get install -y -qq swi-prolog > /dev/null
```

Cell 2 (for every experiment; the first line must be %%writefile):

```
%%writefile ex26.pl
<paste the Prolog program here>
```

Cell 3 (run the program):

```
!swipl -q -s ex26.pl -t halt
```

Ex. No. 36: Best First Search Algorithm

AIM

To write a Prolog program to implement the Best First Search algorithm.

ALGORITHM

Step 1: Start the program.

Step 2: Represent the graph with edge(Node, Node) facts and give each node a heuristic value h(Node, Value).

Step 3: Place the start node in the open list.

Step 4: Take the first node of the open list; if it is the goal, return the path.

Step 5: Otherwise generate its children, add them to the open list and sort the list by heuristic value (lowest first).

Step 6: Repeat steps 4 and 5 until the goal is found.

Step 7: Display the path.

Step 8: Stop the program.

CODE

```
% Experiment 36: Best First Search algorithm (greedy, uses heuristic h(n))
edge(s, a).
edge(s, b).
edge(a, c).
edge(a, d).
edge(b, e).
edge(b, f).
edge(e, h).
edge(f, g).

% Heuristic: estimated distance from each node to the goal g
h(s, 10).
h(a, 7).
h(b, 6).
h(c, 8).
h(d, 5).
h(e, 4).
h(f, 2).
h(g, 0).
h(h, 3).

best_first(Start, Goal, Path) :-
    h(Start, H),
    search([H-[Start]], Goal, RevPath),
    reverse(RevPath, Path).

% Goal reached
search([_-[Goal|T]|_], Goal, [Goal|T]) :- !,
    format("Goal ~w reached.~n", [Goal]).
% Expand the node with the lowest h value
search([H-[N|T]|Rest], Goal, Sol) :-
    format("Expanding node ~w (h = ~w)~n", [N, H]),
    findall(H2-[M,N|T],
            ( edge(N, M), \+ member(M, [N|T]), h(M, H2) ),
```

```
            Children),
    append(Rest, Children, Open),
    keysort(Open, Sorted),                      % lowest h first
    findall(K, member(K-_, Sorted), Keys),
    format("   Open list h-values (sorted): ~w~n", [Keys]),
    search(Sorted, Goal, Sol).

main :-
    format("?- best_first(s, g, Path).~n~n"),
    best_first(s, g, Path),
    format("~nPath = ~w~n", [Path]).

:- initialization(main).
```

Colab run command:

```
!swipl -q -s ex36.pl -t halt
```

OUTPUT

```
?- best_first(s, g, Path).

Expanding node s (h = 10)
   Open list h-values (sorted): [6,7]
Expanding node b (h = 6)
   Open list h-values (sorted): [2,4,7]
Expanding node f (h = 2)
   Open list h-values (sorted): [0,4,7]
Goal g reached.

Path = [s,b,f,g]
```

RESULT

Thus the Prolog program to implement Best First Search was written and executed successfully; the path s - b - f - g was found.

Ex. No. 37: Medical Diagnosis

AIM

To write a Prolog program for medical diagnosis (a simple expert system).

ALGORITHM

Step 1: Start the program.

Step 2: Store the symptoms of each patient as symptom(Patient, Symptom) facts.

Step 3: Write diagnosis rules: a disease is diagnosed when all its required symptoms are present.

Step 4: Store a suitable treatment/advice for each disease.

Step 5: Run the diagnosis query for every patient and display the disease and advice.

Step 6: If no rule matches, report that no disease could be identified.

Step 7: Stop the program.

CODE

```
% Experiment 37: Medical Diagnosis (Expert System)
symptom(ravi,  fever).
symptom(ravi,  cough).
symptom(ravi,  body_pain).

symptom(priya, sneezing).
symptom(priya, runny_nose).
symptom(priya, sore_throat).

symptom(arun,  fever).
symptom(arun,  chills).
symptom(arun,  sweating).

symptom(meena, headache).
symptom(meena, nausea).
symptom(meena, sensitivity_to_light).

symptom(kiran, fever).

diagnosis(P, flu)         :- symptom(P, fever), symptom(P, cough), symptom(P,
body_pain).
diagnosis(P, common_cold) :- symptom(P, sneezing), symptom(P, runny_nose),
symptom(P, sore_throat).
diagnosis(P, malaria)     :- symptom(P, fever), symptom(P, chills), symptom(P,
sweating).
diagnosis(P, migraine)    :- symptom(P, headache), symptom(P, nausea), symptom(P,
sensitivity_to_light).

treatment(flu,         'Rest, fluids and paracetamol; consult a doctor').
treatment(common_cold, 'Rest, warm fluids and steam inhalation').
treatment(malaria,     'Consult a doctor immediately for a blood test and anti-
malarial drugs').
treatment(migraine,    'Rest in a dark, quiet room; consult a doctor').

report(P) :-
    format("?- diagnosis(~w, Disease).~n", [P]),
```

```
    (   diagnosis(P, D)
    ->  treatment(D, T),
        format("Disease = ~w~nAdvice  = ~w~n~n", [D, T])
    ;   format("false. (symptoms do not match any known disease)~n~n")
    ).

main :-
    forall(member(P, [ravi, priya, arun, meena, kiran]), report(P)).

:- initialization(main).
```

Colab run command:

```
!swipl -q -s ex37.pl -t halt
```

OUTPUT

```
?- diagnosis(ravi, Disease).
Disease = flu
Advice  = Rest, fluids and paracetamol; consult a doctor

?- diagnosis(priya, Disease).
Disease = common_cold
Advice  = Rest, warm fluids and steam inhalation

?- diagnosis(arun, Disease).
Disease = malaria
Advice  = Consult a doctor immediately for a blood test and anti-malarial drugs

?- diagnosis(meena, Disease).
Disease = migraine
Advice  = Rest in a dark, quiet room; consult a doctor

?- diagnosis(kiran, Disease).
false. (symptoms do not match any known disease)
```

RESULT

Thus the Prolog program for medical diagnosis was written and executed successfully, and the output was verified. (This program is only a lab exercise and not a substitute for a doctor.)

Ex. No. 38: Forward Chaining

AIM

To write a Prolog program for forward chaining and to incorporate the required queries.

ALGORITHM

Step 1: Start the program.

Step 2: Store the initial facts and the rules of the form rule(Conclusion, [Conditions]).

Step 3: Load the initial facts into the working memory (known/1).

Step 4: Find a rule whose conditions are all known and whose conclusion is not yet known.

Step 5: Fire the rule: add its conclusion to the working memory and print it.

Step 6: Repeat steps 4 and 5 until no new fact can be inferred.

Step 7: Run queries on the working memory.

Step 8: Stop the program.

CODE

```
% Experiment 38: Forward Chaining (data-driven)
:- dynamic known/1.

% Initial facts
fact(has_fur).
fact(gives_milk).
fact(eats_meat).
fact(has_stripes).

% rule(Conclusion, [Conditions])
rule(mammal,    [has_fur, gives_milk]).
rule(carnivore, [mammal, eats_meat]).
rule(tiger,     [carnivore, has_stripes]).
rule(cheetah,   [carnivore, has_spots]).

all_known([]).
all_known([C|Cs]) :- known(C), all_known(Cs).

% Keep firing rules until nothing new can be inferred
forward :-
    rule(Conclusion, Conditions),
    \+ known(Conclusion),
    all_known(Conditions),
    assertz(known(Conclusion)),
    format("Rule fired: ~w  =>  ~w~n", [Conditions, Conclusion]),
    forward.
forward.

ask(Goal) :-
    ( call(Goal) -> R = true ; R = false ),
    format("?- ~w.~n~w~n~n", [Goal, R]).

main :-
    forall(fact(F), assertz(known(F))),
    format("Initial facts: "), findall(F, known(F), L0), format("~w~n~n", [L0]),
    forward,
```

```
    format("~nAll known facts: "), findall(F, known(F), L1), format("~w~n~n",
[L1]),
    ask(known(tiger)),
    ask(known(mammal)),
    ask(known(cheetah)).

:- initialization(main).
```

Colab run command:

```
!swipl -q -s ex38.pl -t halt
```

OUTPUT

```
Initial facts: [has_fur,gives_milk,eats_meat,has_stripes]

Rule fired: [has_fur,gives_milk]  =>  mammal
Rule fired: [mammal,eats_meat]  =>  carnivore
Rule fired: [carnivore,has_stripes]  =>  tiger

All known facts:
[has_fur,gives_milk,eats_meat,has_stripes,mammal,carnivore,tiger]

?- known(tiger).
true

?- known(mammal).
true

?- known(cheetah).
false
```

RESULT

Thus the Prolog program for forward chaining was written and executed successfully; the facts mammal, carnivore and tiger were inferred.

Ex. No. 39: Backward Chaining

AIM

To write a Prolog program for backward chaining and to incorporate the required queries.

ALGORITHM

Step 1: Start the program.

Step 2: Store the known facts and the rules of the form rule(Conclusion, [Conditions]).

Step 3: Start from the goal to be proved.

Step 4: If the goal is a known fact, it is TRUE.

Step 5: Otherwise find a rule whose conclusion is the goal and recursively try to prove every condition of that rule.

Step 6: If all conditions are proved, the goal is TRUE; otherwise it is FALSE.

Step 7: Print the proof steps with indentation and run queries.

Step 8: Stop the program.

CODE

```
% Experiment 39: Backward Chaining (goal-driven)
fact(has_fur).
fact(gives_milk).
fact(eats_meat).
fact(has_stripes).

% rule(Conclusion, [Conditions])
rule(mammal,    [has_fur, gives_milk]).
rule(carnivore, [mammal, eats_meat]).
rule(tiger,     [carnivore, has_stripes]).
rule(cheetah,   [carnivore, has_spots]).

% prove(Goal, Depth) - works backwards from the goal to the facts
prove(G, D) :-
    fact(G),
    Sp is D * 3, tab(Sp),
    format("~w : TRUE (given fact)~n", [G]).
prove(G, D) :-
    rule(G, Conditions),
    Sp is D * 3, tab(Sp),
    format("~w : to prove it we need ~w~n", [G, Conditions]),
    D1 is D + 1,
    prove_all(Conditions, D1),
    tab(Sp),
    format("~w : TRUE (derived)~n", [G]).

prove_all([], _).
prove_all([C|Cs], D) :- prove(C, D), prove_all(Cs, D).

query(Goal) :-
    format("?- prove(~w).~n", [Goal]),
    (   prove(Goal, 0)
    ->  format("Answer: yes~n~n")
    ;   format("Answer: no (cannot be proved from the known facts)~n~n")
    ).
```

```
main :-
    query(tiger),
    query(cheetah).

:- initialization(main).
```

Colab run command:

```
!swipl -q -s ex39.pl -t halt
```

OUTPUT

```
?- prove(tiger).
tiger : to prove it we need [carnivore,has_stripes]
   carnivore : to prove it we need [mammal,eats_meat]
      mammal : to prove it we need [has_fur,gives_milk]
         has_fur : TRUE (given fact)
         gives_milk : TRUE (given fact)
      mammal : TRUE (derived)
      eats_meat : TRUE (given fact)
   carnivore : TRUE (derived)
   has_stripes : TRUE (given fact)
tiger : TRUE (derived)
Answer: yes

?- prove(cheetah).
cheetah : to prove it we need [carnivore,has_spots]
   carnivore : to prove it we need [mammal,eats_meat]
      mammal : to prove it we need [has_fur,gives_milk]
         has_fur : TRUE (given fact)
         gives_milk : TRUE (given fact)
      mammal : TRUE (derived)
      eats_meat : TRUE (given fact)
   carnivore : TRUE (derived)
Answer: no (cannot be proved from the known facts)
```

RESULT

Thus the Prolog program for backward chaining was written and executed successfully; the goal tiger was proved and the goal cheetah could not be proved.

Ex. No. 40: Web Blog using WordPress (Anchor Tag, Title Tag, etc.)

AIM

To create a web blog using WordPress to demonstrate the Anchor tag, Title tag and other HTML tags.

ALGORITHM

Step 1: Open wordpress.com (or a local WordPress installation) and sign up / log in.

Step 2: Create a new site and choose a free theme.

Step 3: Go to Settings > General and enter the Site Title and Tagline. WordPress places the site title and the post title inside the <title> tag of the page.

Step 4: Go to Posts > Add New and enter the Post Title.

Step 5: Add a "Custom HTML" block and paste the HTML code given below (anchor tags with href and title attributes, heading tags, lists, image tag).

Step 6: Click Preview to check the page, then click Publish.

Step 7: Open the blog URL, click the links, hover over them to see the title tooltips and verify the output.

Step 8: Stop.

CODE

```
<h1>My AI Lab Journal</h1>
<h2>Welcome to my blog</h2>
<p>This blog is created using <b>WordPress</b> to demonstrate HTML tags such as
the <i>anchor tag</i>, <i>title tag</i>, heading tags, lists and images.</p>

<h3>1. Anchor Tag (Hyperlinks)</h3>
<p>External link: <a href="https://wordpress.org" title="Visit the official
WordPress website" target="_blank">Visit WordPress.org</a></p>
<p>Email link: <a href="mailto:student@example.com" title="Send me an
email">Contact the author</a></p>
<p>Internal link: <a href="#topics" title="Jump to the topics list">Go to Topics
section</a></p>

<h3>2. Title Attribute</h3>
<p>Hover the mouse over this link to see the tooltip text: <a
href="https://www.python.org" title="Python Programming Language">Python</a></p>

<h3 id="topics">3. Topics Covered in CSA 17</h3>
<ul>
  <li>Python programming basics</li>
  <li>Search algorithms (BFS, DFS, A*)</li>
  <li>Prolog and expert systems</li>
</ul>

<h3>4. Ordered List</h3>
<ol>
  <li>Install and learn Python</li>
  <li>Write AI programs</li>
  <li>Test them in Google Colab</li>
</ol>
```

```
<h3>5. Image Tag</h3>
<img src="data:image/svg+xml;utf8,<svg xmlns='http://www.w3.org/2000/svg'
width='300' height='120'><rect width='300' height='120' fill='%23dbe7f5'/><text
x='150' y='70' font-size='28' text-anchor='middle' fill='%231a3c6e' font-
family='Arial'>AI Lab</text></svg>" alt="AI Lab banner image" width="300">

<hr>
<p>Posted by: <b>Student</b> | Category: <i>Artificial Intelligence</i></p>
```

OUTPUT

Rendered output of the blog post:

My AI Lab Journal

Welcome to my blog

This blog is created using **WordPress** to demonstrate HTML tags such as the *anchor tag*, *title tag*, heading tags, lists and images.

1. Anchor Tag (Hyperlinks)

External link: Visit WordPress.org

Email link: Contact the author

Internal link: Go to Topics section

2. Title Attribute

Hover the mouse over this link to see the tooltip text: Python

3. Topics Covered in CSA 17

- Python programming basics
- Search algorithms (BFS, DFS, A*)
- Prolog and expert systems

4. Ordered List

1. Install and learn Python
2. Write AI programs
3. Test them in Google Colab

5. Image Tag



Posted by: **Student** | Category: *Artificial Intelligence*

RESULT

Thus the web blog was created using WordPress, and the Anchor tag, Title tag, heading tags, lists and image tag were demonstrated successfully.